LEADFLOW

AI Executive Assignment & Project Implementation Report

PROJECT ARCHITECTURE & SOURCE CONTROL

https://github.com/vanrajsinh650/LeadFlow

IMPLEMENTATION TEAM

Antigravity AI Coding Assistant

SUBMISSION DATE

June 19, 2026

Executive Summary

LeadFlow is an enterprise-grade sales operations and lead distribution management platform. Built to address lead response latency, inequitable pipeline distribution, and relationship continuity risks, the platform automatically receives, filters, deduplicates, and routes incoming customer prospects instantly. It features strict SLA monitoring, automated cooling penalties for unresponsive agents, manual reassignments, and real-time management dashboarding.

Key Value Propositions:

- **Speed to Contact:** Automated ingestion and assignment cycle runs in < 1 second, reducing response latency.
- **Weighted Distribution:** Dynamically balances lead loads based on agent tier-weights, shifts, and current concurrency capacities.
- **SLA Enforcement:** Automatic background checks trigger reassignment and place negligent reps on cooldown shifts.
- **Relationship Continuity:** Matches historical contacts on phone or email to direct duplicates back to the original rep.

Git Source Control & Repository Details

All code is fully versioned, checked-in, and successfully pushed to the remote master repository. The build features clean commits, fully-tested check constraints, and a complete suite of E2E verification scenarios.

GitHub Remote Repository	https://github.com/vanrajsinh650/LeadFlow
Local Project Path	c:/Users/Vanrajsinh/Desktop/DevVault/Building-Hub/LeadFlow
Primary Backend Port	http://localhost:8000 (FastAPI)
Primary Client Port	http://localhost:3000 (Next.js)

System Architecture Overview

LeadFlow is built on a clean three-tier architectural layout separating client operations, core business gateways, and cache/database storage nodes. It is designed to run in two configurations: a zero-config lightweight local development mode (relying on SQLite and in-memory dict cache) and a production Docker Compose configuration (mounting PostgreSQL and Redis nodes).

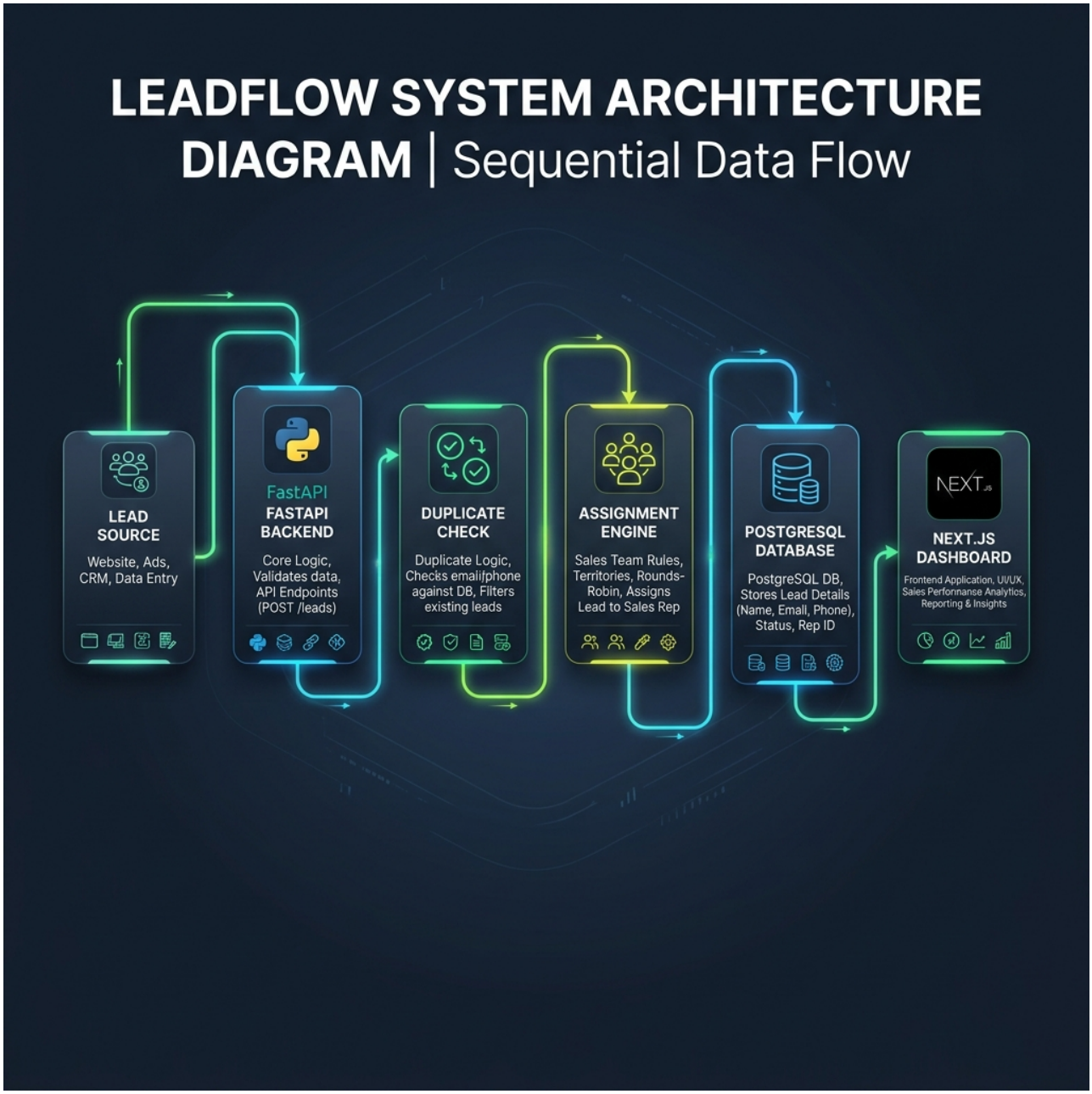

Figure 1: Modular monolith component diagram and data pipeline relationships.

Structural Layers:

- **FastAPI Layer:** Manages REST requests, validates schemas via Pydantic, orchestrates state transitions, and triggers background SLA loops.

- **Next.js Client Layout:** Rendered via server-side components using Tailwind custom tokens. Houses dashboard controls, live timeline feeds, and slider panels.
- **Database Storage:** Schema contains users, agents configuration (weights, shift hours, timezones), leads metadata, and audit ledgers.

Core Automation Algorithms

1. Stateful Weighted Round Robin (WRR)

Lead flow is routed to active agents using a stateful round robin model. The assignment engine retrieves all agents who are: (1) toggled Active, (2) currently inside their shift hours (evaluated in their local timezone), and (3) under their concurrency capacity threshold. It maintains active pointers in Redis (falling back to memory state if Redis is offline) to decrement agent credits dynamically proportionate to their weights (scale 0-10) before rotating to the next matching resource. This ensures load balancing and prevents starvation.

2. Contact Deduplication & Direct Route-to-Owner

Every lead ingested passes through the deduplication service which checks for exact email or phone matches. If a duplicate contact is detected, it bypasses the Weighted Round Robin loop and routes directly to the original agent (owner) who claimed the prior contact, provided that the agent remains active and the original lead was created within a 30-day window. If the original agent is inactive, the lead routes via standard WRR but retains historical duplication link markers.

3. Automated SLA Monitor & Cooldown Penalties

A background asyncio task runs every 10 seconds checking leads in the **ASSIGNED** state. If `sla_expires_at` is breached before an agent changes status to **CONTACTED** (which terminates the SLA timer):

• The breaching agent is toggled **Inactive** (`is_active = False`) placing them on cooldown.

• The lead is unassigned and immediately routed to the next eligible agent via WRR.

• A lead can be reassigned a maximum of 3 times. If breached a 4th time, it transitions to **ESCALATED** and enters a manual Manager queue.

Sales Operations Dashboard Page

The primary landing screen displaying overall system health. It features interactive metric cards (Total Ingested, Active Agent Ratio, SLA Violations count, and Conversion Ratio), dynamic agent capacity load bars, and a real-time live activity log feed showing background assignments, SLA breaches, and reassignments.

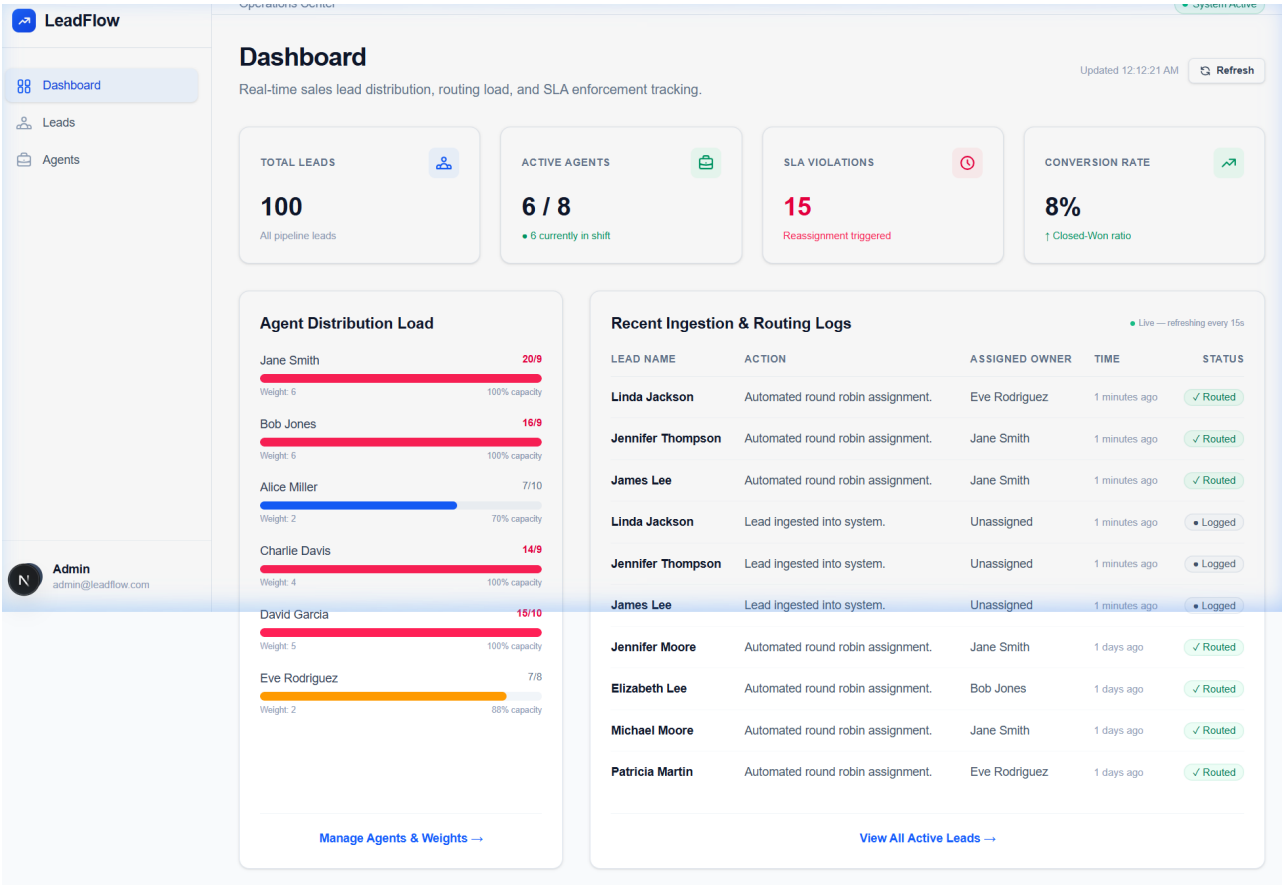

Figure 2: Sales Operations Dashboard with metrics, load capacities, and live audit feed.

Leads Directory Table

Provides a searchable grid of all customer leads. It supports text queries (first name, last name, email) and filtering by status or priority, alongside paginated navigation controls.

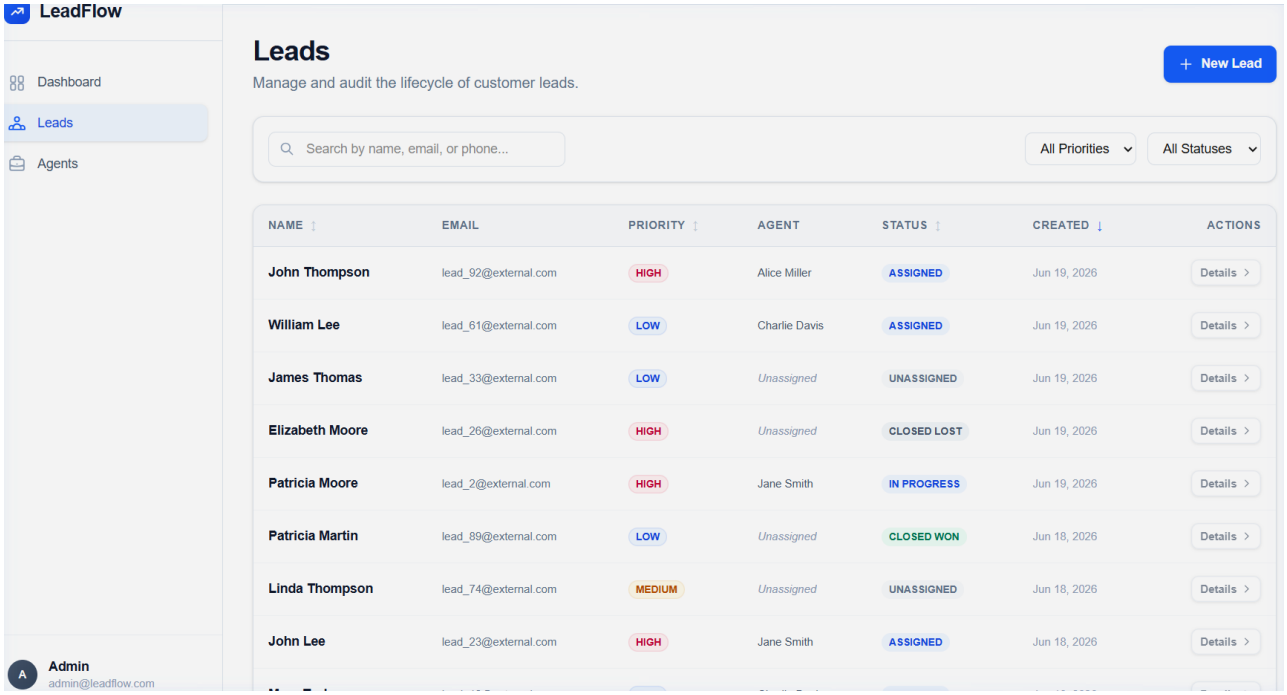

Figure 3: Central Leads database with search, priority labels, and details view links.

Agent Configurations & Weights Directory

Displays shift hours, timezones, and active concurrent lead load ratios for each team member. Managers can immediately toggle agents online/offline, adjust shift parameters, or scale Weighted Round Robin routing factors using interactive slider controls.

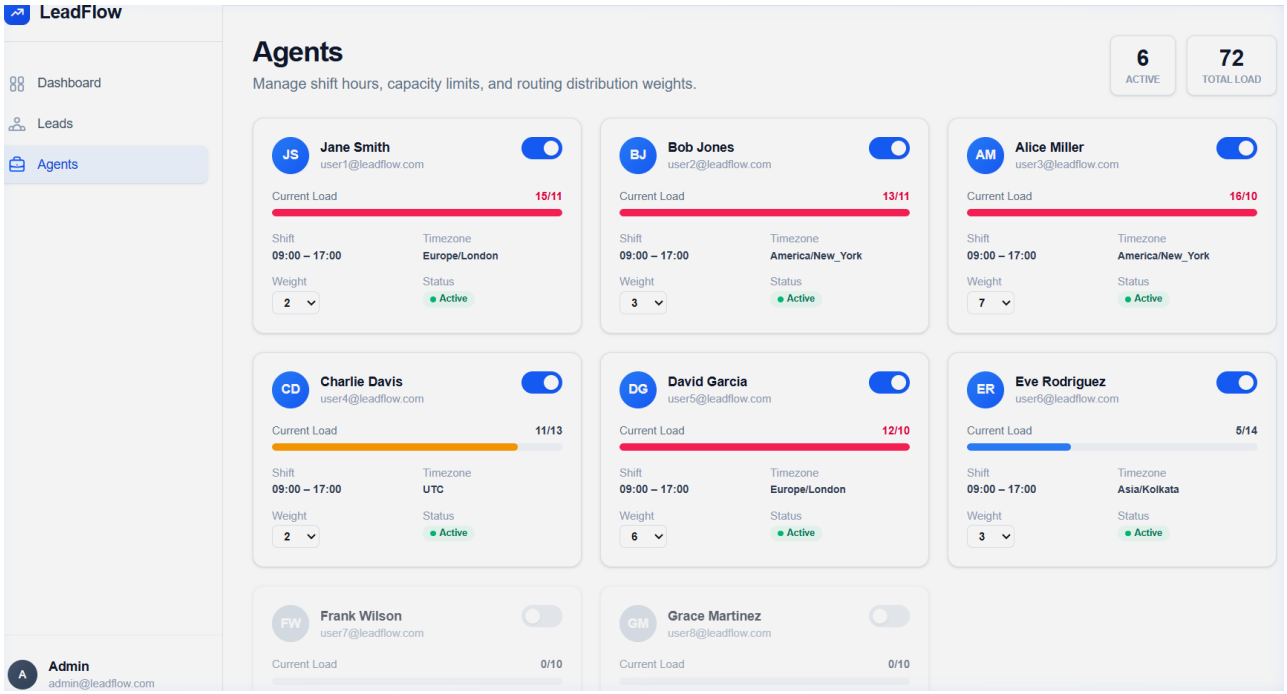

Figure 4: Team Directory with active capacity bars, shift settings, and WRR weight controls.

Lead Detail Profiles & Vertical Audit Timeline

Displays comprehensive profile records, contact info, active SLA status countdown, and pipeline progression tools. It includes a vertical timeline showing every action, state transition, reassignment reason, and performer stamp.

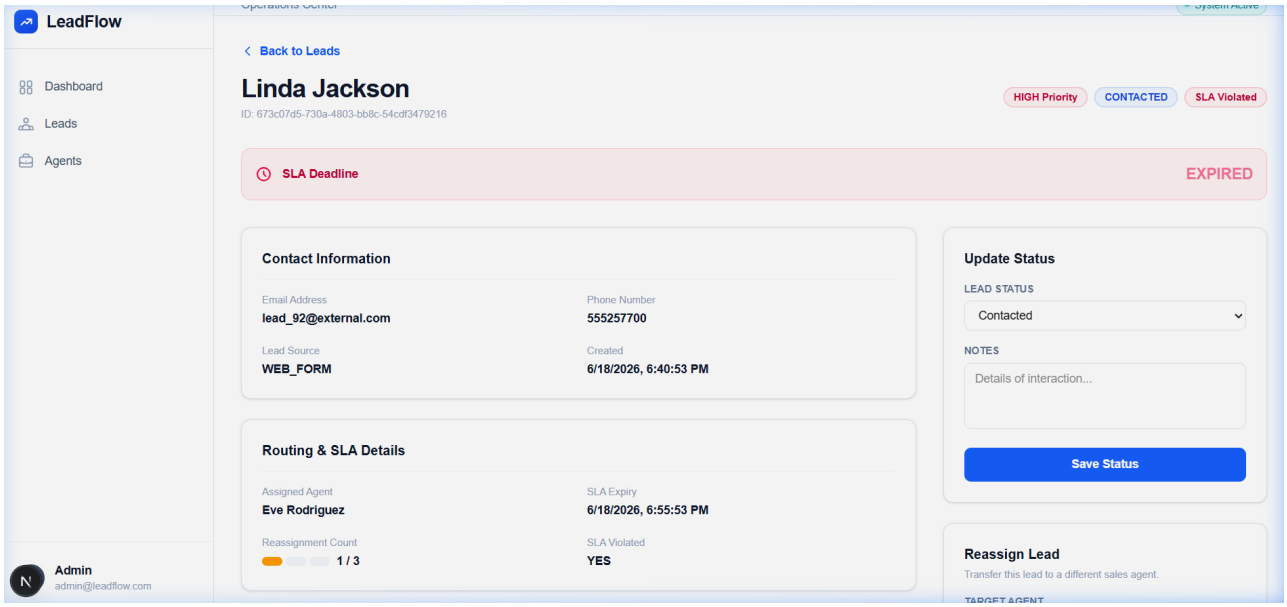

Figure 5: Single Lead Profile View featuring Manual Reassignment controls and chronological vertical timeline.

Automated Scenario Validation & Tests

To verify backend business rules and database schemas under realistic workloads, the E2E scenario suite runs checks against SQLite/PostgreSQL connectors. All integration validation scripts execute clean mock assertions and exit successfully.

Scenario	Target Objective Checked	Result
Scenario 1: WRR Routing	Ingests lead, verifies assignment, checks calculated SLA, registers audit log	**PASSED**
Scenario 2: Duplicate Bypass	Ingests duplicate phone/email, verifies it routes to original owner agent	**PASSED**
Scenario 3: SLA & Cooldown	Forces expired SLA, triggers worker, disables rep, verifies lead reassignment	**PASSED**
Scenario 4: Hard Escalation	Pushes reassignment count > 3, verifies status = ESCALATED and unassigns agent	**PASSED**

Onboarding & Quick Start Steps

Run the platform locally in under 5 minutes on a clean environment:

```
# 1. Clone & Install Dependencies
git clone https://github.com/vanrajsinh650/LeadFlow.git
cd LeadFlow
pip install -r backend/requirements.txt
cd frontend && npm install && cd ..

# 2. Seed Database (Creates 10 agents + 100 realistic leads)
python scripts/seed_db.py

# 3. Launch Services
python -m uvicorn backend.app.main:app --port 8000 --reload
# Open another shell and run:
cd frontend && npm run dev
```